



CAPSTONE PROJECT

ResolveAI

The AI Travel Support Desk for SkyNova Airlines

Design, build, evaluate and deploy an autonomous, policy-grounded support agent — a conversational assistant fused with a multi-agent decision system. Prompt the future of travel support.

Stack: Python · Gemini · Google ADK · LangChain · RAG · MCP · Google Cloud

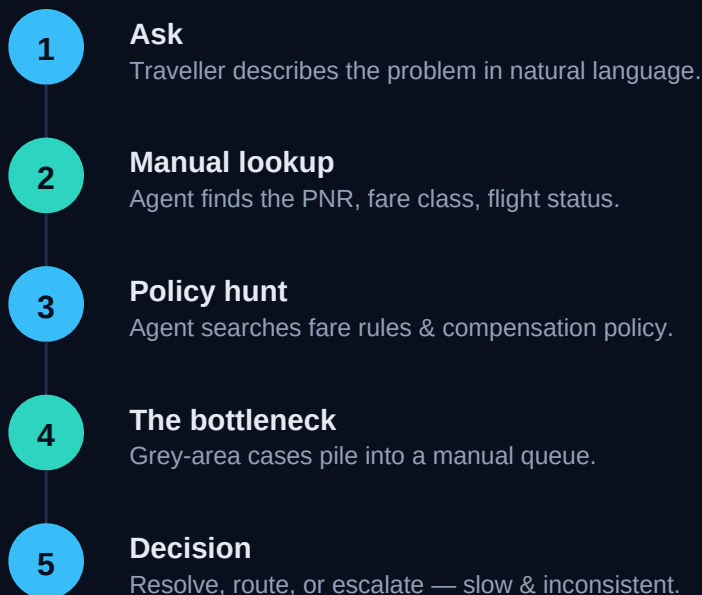
BACKGROUND

Context

In modern travel, the support desk is where customer patience goes to die. A traveler arrives with intent expressed in natural language — “my flight got delayed five hours, do I get a refund?”, “cancel SN-204 and refund me,” “my bag never arrived.” Today a human agent must interpret that request, look up the booking (PNR), find the correct fare rule (a Saver ticket and a Flex ticket carry completely different refund, change and baggage rights), weigh timing and value, and only then decide: answer it now, route it to the right team, or escalate to a senior agent.

This linear, manual process introduces significant latency and inconsistency. Rigid auto-rules wrongly reject travellers who are genuinely owed a refund; an over-cautious desk pushes everything into a human queue whose cost scales linearly with passenger volume. The result is friction at the exact moment that decides loyalty — and direct margin leakage.

The status-quo journey (the “before”):



By replacing this with an AI Travel Support Desk, we bridge the gap between a static query-response chatbot and a senior human reviewer. The assistant understands nuance, maintains context across turns, grounds every claim in real policy, and — when a genuine decision is needed — hands the case to a multi-agent triage pipeline that adjudicates and explains itself. Minimising friction here protects both customer lifetime value and margin.

THE CHALLENGE

Problem Statement

Design and implement **ResolveAI**, an AI Travel Support Desk for SkyNova Airlines that behaves like an experienced support agent — at the speed of an API call. Your solution must do two things as one product:

1 · Converse & resolve

A conversational assistant that parses a natural-language request, looks up the traveller's **actual booking**, and answers **strictly from SkyNova's knowledge base and fare/policy documents** — never a hallucinated refund amount, fee, or baggage allowance. It must maintain **session state** so follow-ups (*"and my return leg?"*) work without repeating the PNR, and must ask a clarifying question when a required detail is missing.

2 · Triage & decide

When a request needs a decision the assistant cannot safely make alone — a refund, delay compensation, or a baggage claim — it must hand the case to a **multi-agent triage pipeline** that reads the booking, checks the fare rule/policy via RAG, scores value and risk, and returns a final verdict: **Auto-Resolve, Route-to-Team, or Escalate-to-Human** — with an auditable **justification trail** citing the exact policy clause and signals, plus a summarised dossier when it escalates.

Your core technical challenge is that the system must function as a reliable **agent loop** and a **coordinated multi-agent system** — not a single prompt. Every case action (look up a PNR, open a case, update it, assign a team, escalate) must flow through **one shared Booking & Case system exposed over MCP**, reused by both the assistant and the orchestrator. Finally, the system must be **deployed to Google Cloud**.

WHAT IT MUST DO

Technical Requirements

- **Intent & entity extraction:** use an LLM with few-shot prompting to map free text into a structured request (issue type, PNR, passenger, flight/leg, fare class, amount, urgency).
- **Booking & policy grounding (RAG):** build a retrieval pipeline (sentence embeddings + a vector store such as Chroma/FAISS) over the knowledge base and fare/policy documents; answer only from retrieved records; use metadata filtering to fetch rules by fare class.
- **Agentic tooling:** design the assistant as an agent loop (ReAct via ADK/LangChain) with tools — `search_policy()`, `get_booking()`, `open_case()`, `get_status()`. The LLM decides which tool to call and when.
- **Multi-agent triage:** orchestrate specialised agents (Intent, Policy-RAG, Risk) under an orchestrator that produces a structured verdict + justification. Not a single prompt.
- **Grounding & anti-hallucination:** responses about price, refund, fee, availability or coverage must be traceable to a retrieved record; ask a clarifying question when a required filter is missing.
- **Session management:** maintain short-term state so refinements and coreference (“the return one”) resolve across turns.
- **Shared tools over MCP:** expose the Booking & Case system as an MCP server reused by the assistant and the orchestrator (write once, reuse everywhere).
- **Guardrails:** input validation, PII redaction (PNR, card, contact), step/cost caps, and human-in-the-loop escalation as the ultimate safety valve.
- **Deployment:** containerise and deploy to Google Cloud Run.

WHAT YOU SUBMIT

Deliverables

- **Working prototype:** a web UI (traveller chat + a reviewer view of the escalation queue) demonstrating resolve, refine, clarify, and triage.
- **MCP case server:** the Booking & Case system, shown being called by both the assistant and the orchestrator.
- **Architecture diagram:** the agent loop, the multi-agent triage, the RAG pipeline, the MCP server, and session state.
- **README:** prompt engineering, tool design, RAG design, and dependency/deploy instructions.
- **Transcript samples:** 3–5 annotated conversations showing query refinement, coreference, grounding, and an escalation with its dossier.
- **Evaluation report:** the metrics below computed on a small labelled set.
- **Failure analysis:** edge cases (ambiguous intent, sarcastic reasons, policy edge cases, missing PNR) where the system underperforms or fails to ground.
- **Live URL:** the deployed Google Cloud endpoint.

Example Solution Approach

- Parse the request into structured fields with a few-shot extraction prompt (issue, PNR, fare class, amount).
- Embed the KB + fare/policy docs; store in a vector index; retrieve top-k, filtered by fare class where relevant.
- Run an agent loop with tools; the assistant answers grounded, or opens a case via the MCP server.
- For decisions, run the orchestrator: Intent → Policy(RAG) → Risk (run the specialists in parallel), then combine into a verdict + justification.
- Reach a verdict (Auto-Resolve / Route / Escalate); on escalation, auto-write a dossier for the human queue.
- Maintain short-term session memory so refinements work across turns.

BUILD WITH

Suggested Tech Stack

Python 3.11 · an LLM (Google **Gemini** — free key, no credit card; or a local model via Ollama) · **Google ADK** or LangGraph/LangChain for the agent loop · **Chroma** or FAISS for vector search · sentence-transformer / Gemini embeddings · **FastMCP** for the Booking & Case server · **Gradio** or FastAPI + a simple frontend for the UI · Docker → **Google Cloud Run** for deployment.

Development Data (mock — generate it)

No real airline data is required. Generate a self-contained SkyNova dataset:

- **Knowledge base:** 15–20 short articles (baggage, web check-in, seats, add-ons, flight status).
- **Fare rulebooks:** three fare classes — Saver, Flex, Business — with different refund, change-fee, baggage and seat rights.
- **Policy docs (RAG corpus):** refund & cancellation, delay/cancellation compensation, baggage (lost/damaged), special assistance.
- **Bookings dataset:** ~200–500 synthetic PNRs (passenger, flight, route, date, fare class, price, status, history).
- **Free-text reasons:** synthesise with varied tone (normal, evasive, sarcastic, contradictory) to stress-test grounding.

HOW IT'S SCORED

Evaluation Dataset

Author a golden set of 50–100 scenarios by hand from your catalogue — the most signal-rich eval you will have. Cover: a simple policy question; a multi-turn refinement (“return leg too”); an ambiguous query needing a clarifying question; a refundable case that should auto-approve; a non-refundable dispute that should escalate; a delay-compensation claim; a baggage claim; and an abuse signal (repeat-refund / chargeback risk).

Evaluation Criteria

- **Resolution rate** — % of requests correctly closed by the assistant without a human.
- **Routing accuracy** — % of cases sent to the correct team (Refunds / Rebooking / Baggage / Special Assistance).
- **Escalation calibration** — high coverage of truly-hard cases escalated + high automation on clear ones (target: most clear cases auto-decided).
- **Grounding / hallucination rate** — % of factual claims (refund, fee, allowance, compensation) traceable to a retrieved record (target: <2%).
- **Justification faithfulness** — does the reasoning trail cite the real fare clause + signals? Scored 0–3 (generic → grounded + policy-cited).
- **Orchestration reliability** — % of runs where all agents complete, tools fire in the correct order, and a structured verdict is produced in time.

This capstone is designed the spec-driven way: understand the problem → write the spec → design the architecture → build it with AI (Antigravity / Claude) → evaluate → deploy → present. You don't start by writing code. You start by understanding the problem. The AI writes the code — you own the thinking.